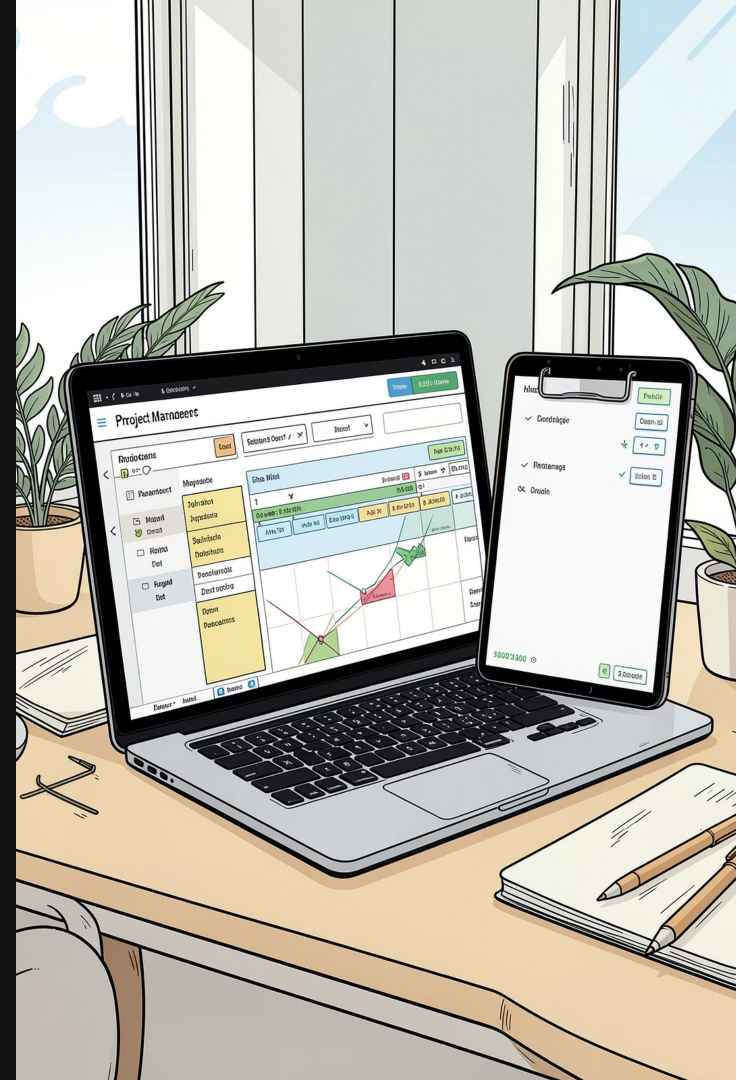


Project Management System — Mini Project

A concise technical overview and documentation of a web-based Project Management System built with a Java 17 + Spring Boot full-stack architecture. Designed for Admin, Manager and Employee roles to streamline project planning, task assignment, progress tracking and reporting.

1. Submitted by: **G . Chaithanya Varaprasad** | Roll No: **23AK1A3608**
2. Submitted by: **G . Punith Kumar** | Roll No: **23AK1A3629**



Agenda

Scope What the system does and who it serves	Tech Stack Backend, frontend, database and tools	Objectives & Modules Core goals and feature modules	Architecture High-level components and data flow
Sample Workflows Common user journeys	Deployment & Tools How to build, test and run	Next Steps Extensions and recommended improvements	

Project Overview & Users

The system is a web application to manage projects, assign and monitor tasks, and measure team performance. It supports three roles:

- Admin — user management, project creation, reporting
- Manager — assign tasks, track progress, update project status
- Employee — view tasks, update status, submit work updates



Technology Stack



Backend: **Java 17**, **Spring Boot** (REST APIs), **Spring Security** (RBAC), **Spring Data JPA** (persistence).

Frontend: **HTML**, **CSS**, **Bootstrap 5**, **JavaScript** (vanilla).

Database: **MySQL 8.0**.

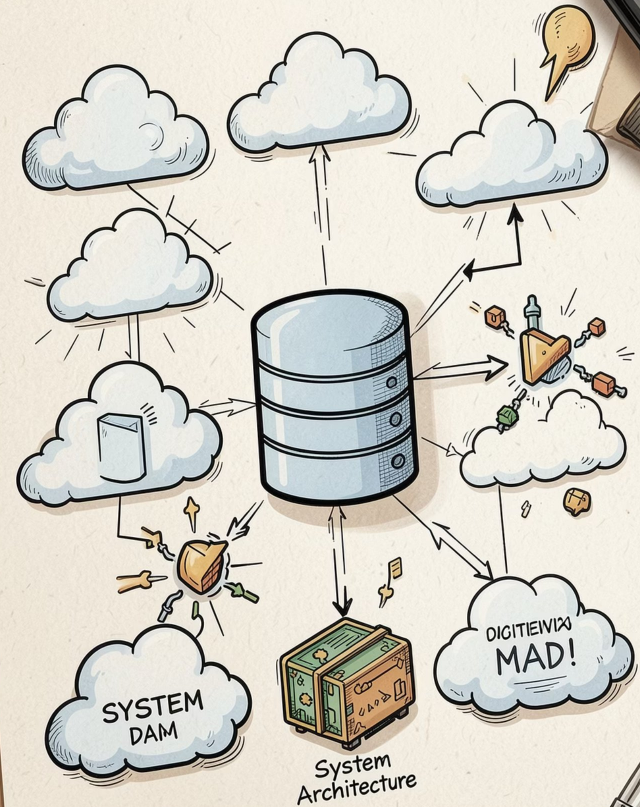
Tools: **Maven**, **MySQL Workbench**, **Postman**, **GitHub**.

☐ Use Java 17 LTS for stability and modern language features.

High-Level Architecture

The application follows a layered architecture:

1. Client (Browser) — UI built with Bootstrap and JS calling REST endpoints
2. API Layer — Spring Boot controllers exposing REST services with JSON
3. Service Layer — Business logic, validation and authorization checks
4. Data Layer — Spring Data JPA repositories mapping to MySQL
5. Security — Spring Security with role-based access control (Admin, Manager, Employee)



Core Objectives



Efficient Project Management

Create and structure projects, milestones and timelines



Task Assignment

Assign tasks to employees with deadlines and priorities



Progress Tracking

Visualize task and project status; update progress in real time



Reporting & Dashboards

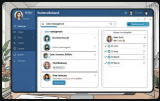
Generate summaries for stakeholders and performance metrics



Role-Based Access

Secure endpoints and UI flows based on user roles

Modules & Key Features



Admin Module

Manage users, roles, permissions; create and archive projects; generate system reports.



Manager Module

Assign tasks, set priorities and deadlines, monitor team progress, update project status.



Employee Module

View personal tasks, update task status, submit work updates and attachments.

Typical Workflows (Diagrams)



Example workflow: Manager creates and assigns a task → Employee updates status and uploads progress → The system logs updates and triggers notifications / dashboard refresh for stakeholders.

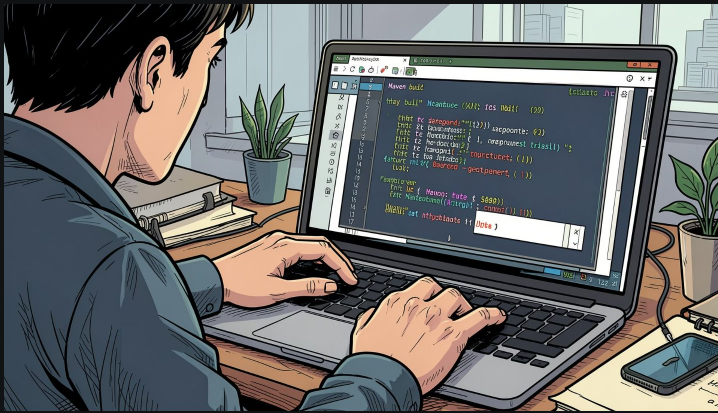
Create Project

Initialize release and
scope

Execute

Work items, update
status

Development, Testing & Deployment



Build: Use **Maven** for dependency management and packaging (jar).

Local Dev: Run Spring Boot app with profile for local MySQL, configure application.properties for datasource.

API Testing: Use **Postman** for endpoint validation (auth flows, CRUD operations).

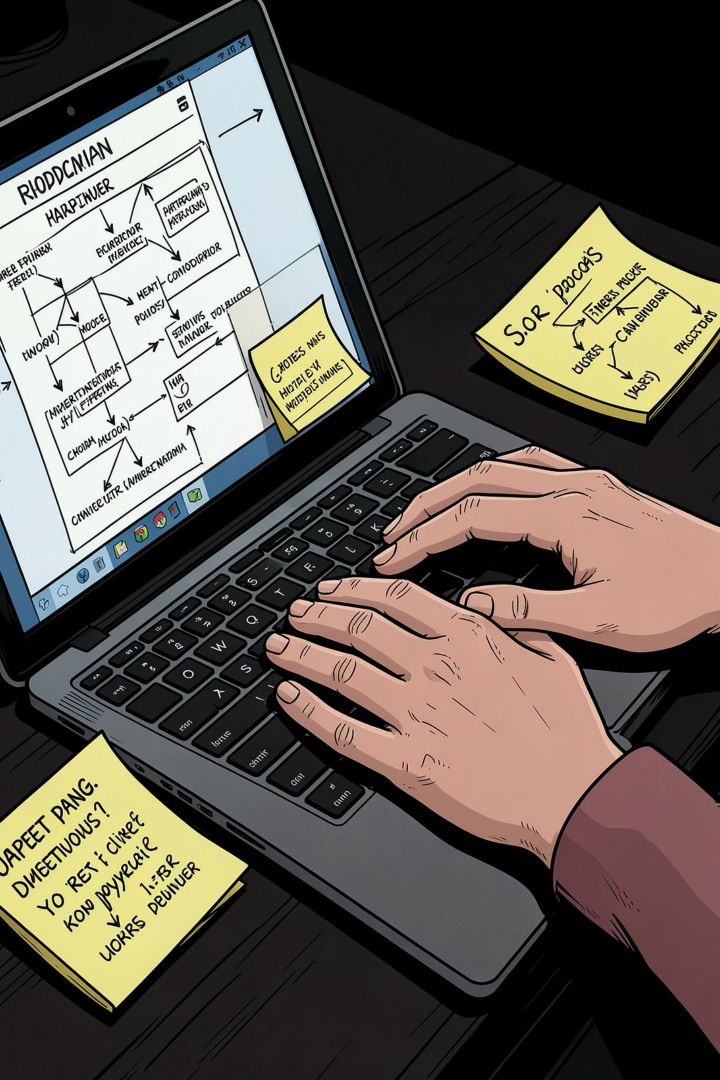
Database: Model schema in **MySQL Workbench**; apply migrations or use Spring JPA auto-ddl in early stages.

Version Control: Host source on **GitHub**, use branches for features and pull requests for reviews.

Next Steps & Recommendations

- Implement pagination and filtering for large project/task lists to maintain UI performance.
- Add audit logs and activity history for compliance and traceability.
- Enhance security: OAuth2 / JWT token flow and secure password policies.
- Introduce dashboards with charts (task completion, velocity) and exportable reports (CSV/PDF).
- Plan integrations: Slack/email notifications, CI/CD for automated builds and deployments.

❑ Ready for production: prepare staging environment and run security & performance testing before go-live.



Thank You

Thank you for your attention. Questions?